

Day 2 - Control Flow: Conditionals, Loops, Comprehensions

Day 2 — Control Flow: Conditionals, Loops, Comprehensions

Objectives

- Understand `if/elif/else` — how your program makes decisions
- Understand that a `for` loop's real job is "go through each thing in this collection," not "count from one number to another" (counting is just the most common special case)
- Understand `while` loops, and when to reach for one instead of a `for` loop
- Understand `break`, `continue`, and the unusual `for...else`
- Write your first list/dict/set "comprehensions" — a compact way to build a new collection from an existing one
- Meet Python 3.10's `match` statement, a modern alternative to a long chain of `if/elif`

Why do programs need "control flow" at all?

So far (Day 1) every example ran every line, in order, exactly once, top to bottom. Real programs need to make decisions ("if the user is over 18, allow access; otherwise don't") and repeat work ("do this once for every item in this list of 500 orders"). **Control flow** is the umbrella term for the instructions that let your program skip lines, repeat lines, or choose between different lines to run, instead of always running everything in a straight line from top to bottom. `if`, `for`, and `while` are the three basic control flow tools you'll use constantly, starting today.

`if` / `elif` / `else` — making decisions

```
age = 20

if age >= 18:
    print("You can vote.")
else:
    print("You cannot vote yet.")
```

Read this exactly as English: "if `age` is greater than or equal to 18, print the first message; otherwise (`else`), print the second." The colon `:` at the end of the `if` line, and the indentation (from Day 0) on the lines below it, are what tell Python "these indented lines belong to this decision." Only one branch ever runs, never both.

When you have more than two possibilities, use `elif` (short for "else if") for each additional check, and Python checks them in order, top to bottom, stopping at the first one that's true:

```
score = 72

if score >= 90:
```

```

    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"

print(grade)  # "C" -- because 72 >= 70 was the first true condition it reached

```

Notice `elif` only gets checked if every condition *above* it was false — once `score >= 70` is found true, Python entirely skips the `else` and moves on past this whole block, without ever checking whether `score >= 90` twice or looking further.

Comparing values

The comparison operators you'll use inside `if` conditions: `==` (equal to — note: two equals signs, since a single `=` means "assign," as you learned Day 1), `!=` (not equal to), `>`, `<`, `>=`, `<=`. You combine multiple conditions with the words `and`, `or`, and `not`:

```

if age >= 18 and has_id:
    print("allowed in")

if is_weekend or is_holiday:
    print("no work today")

if not is_raining:
    print("no umbrella needed")

```

Python also lets you chain comparisons in a way that reads naturally and many other languages don't allow directly:

```

if 0 <= x < 10:
    print("x is between 0 and 9, inclusive-exclusive")

```

This is exactly equivalent to `0 <= x and x < 10`, just more compact and closer to how you'd write it in a math class.

The ternary — an `if` that produces a value

Sometimes you just want to pick between two *values*, not two whole blocks of instructions. Python has a compact one-line form for exactly this:

```

status = "adult" if age >= 18 else "minor"

```

Read it as: "the value is `'adult'` IF `age >= 18`, ELSE the value is `'minor'`." This is called a **ternary expression** (or "conditional expression"). Use it only when each branch is a single simple value — if either side needs more than one step of logic, use a full `if/else` block instead for readability.

`for` loops — repeating an action once per item

Here is the most important reframing in today's lesson: **a `for` loop's job is to walk through a collection of things, one at a time, running the same block of code for each one.** It does NOT fundamentally mean "count from number A to number B" — that's just the single most common thing people use it for, via a helper

called `range` (explained below).

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)
```

Read this as: "for each `fruit` found inside `fruits`, do the following." The loop runs its indented body once per item — three times here — each time with `fruit` pointing at the next item in the list, in order: first `"apple"`, then `"banana"`, then `"cherry"`. Once it's gone through every item, the loop ends automatically and the program continues after it.

You can loop over the individual characters of a string the exact same way, since a string is really just an ordered sequence of characters:

```
for character in "abc":
    print(character)    # prints a, then b, then c, on separate lines
```

`range()` — the tool for "count from A to B"

When you genuinely do want to count (run something exactly 5 times, or go from 1 to 100), Python provides `range(...)`, which produces a sequence of numbers to loop over:

```
for i in range(5):
    print(i)            # prints 0, 1, 2, 3, 4 -- five numbers, STARTING AT 0, and stopping BEFORE 5
```

This surprises almost every beginner at least once: `range(5)` produces `0, 1, 2, 3, 4` — five numbers total, but stopping *before* reaching 5, not including it. This "starts at 0, stops just before the number you gave it" behavior is consistent across many parts of Python (you'll see it again with list slicing on Day 3), so it's worth internalizing now rather than re-deriving it every time.

`range` also accepts a start and a step: `range(2, 10, 2)` produces `2, 4, 6, 8` (starts at 2, stops before 10, counting by 2s each time).

One more thing worth knowing, even though it won't change how you use `range` day to day:

`range(10_000_000)` does *not* actually create ten million numbers in memory all at once — it's a special, lightweight object that produces each number on demand, only as the loop asks for the next one. This matters for performance on huge ranges, and it's your very first preview of an idea called "laziness" that you'll meet properly on Day 10.

Looping over a dictionary

You'll properly meet dictionaries on Day 3, but since you'll see this pattern constantly, here's a preview: looping over a dict by default gives you its keys, one at a time, not its values:

```
scores = {"Ana": 90, "Bo": 85}

for name in scores:
    print(name)          # prints "Ana", then "Bo" -- the KEYS

for name, score in scores.items():
    print(name, score)    # prints "Ana 90", then "Bo 85" -- BOTH key and value, via .items()
```

`enumerate()` — when you need the position AND the item

If you need to know *where* you are in a list while also having the item itself, resist the beginner instinct to write `for i in range(len(fruits))` and then index in with `fruits[i]` — Python has a cleaner, more idiomatic built-in for exactly this:

```
for index, fruit in enumerate(fruits):
    print(index, fruit)    # 0 apple / 1 banana / 2 cherry
```

`zip()` — walking two lists together

If you have two related lists and want to process them in lockstep (item 1 from each together, then item 2 from each together, and so on), `zip` does this for you, stopping automatically once the shorter list runs out:

```
names = ["Ana", "Bo"]
ages = [30, 25]

for name, age in zip(names, ages):
    print(name, age)    # "Ana 30" then "Bo 25"
```

`while` loops — repeat until a condition becomes false

A `for` loop is for "do this once per item in a known collection." A `while` loop is for "keep doing this for as long as some condition stays true" — useful when you don't know in advance how many times you'll need to repeat something.

```
count = 0
while count < 3:
    print("still going:", count)
    count += 1    # shorthand for: count = count + 1
print("done")
```

This prints "still going: 0", "still going: 1", "still going: 2", then stops, because once `count` becomes 3, the condition `count < 3` becomes false and the loop ends. **If you forget to update `count` inside the loop, this becomes an infinite loop** — the condition never becomes false, and the program will run forever (or until you stop it with Ctrl+C in the terminal). This is an extremely common beginner mistake; if your program seems to hang and never finish, an infinite loop is the first thing to suspect.

A rough rule of thumb for choosing between them: **if you know exactly what collection you're going through, use `for`**. **If you're waiting for some condition to change (user input, a value crossing a threshold, "keep trying until it works"), use `while`**.

`break` and `continue` — early exits from a loop

`break` immediately stops the loop entirely, skipping any remaining items:

```
for number in [1, 2, 3, 4, 5]:
    if number == 3:
        break    # stop the loop the instant we hit 3
    print(number)    # prints only 1, then 2
```

`continue` skips just the *rest of the current pass* through the loop body and moves on to the next item, without stopping the whole loop:

```
for number in [1, 2, 3, 4, 5]:
    if number == 3:
        continue
```

```

        continue          # skip printing 3 specifically, but keep looping
    print(number)         # prints 1, 2, 4, 5 -- everything except 3

```

The unusual `for...else`

Python has an `else` clause you can attach to a loop, which is genuinely unusual among programming languages and confuses even people with experience in other languages the first time they see it. It runs only if the loop finished normally, **without** being stopped early by a `break`:

```

numbers = [1, 3, 5, 7]

for n in numbers:
    if n % 2 == 0:          # % is the "modulo" operator -- remainder after division
        print("found an even number:", n)
        break
else:
    print("no even numbers found") # this runs, since the loop never hit `break`

```

This is most useful for exactly this "search for something, and if you never find it, do X" pattern. It's used somewhat rarely in real code because many people find it non-obvious to read — but you should be able to recognize it when you encounter it in someone else's code.

Comprehensions — a compact way to build a new list from an old one

A **list comprehension** is not a new kind of loop — it's a compact expression that builds a brand new list by describing what you want to do to each item of an existing collection, all on one line. It's easiest to learn by comparing the long way and the short way side by side:

```

# the long way, using a full for loop
squares = []
for n in range(10):
    squares.append(n * n)

# the exact same result, as a list comprehension
squares = [n * n for n in range(10)]

```

Read a comprehension **right to left**, roughly as: "for each `n` in `range(10)`, give me `n * n`, and collect all those results into a new list." Both versions above produce the identical list `[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]` — the comprehension is just a shorter way to write the same idea once it's familiar.

You can also filter which items are included, by adding an `if` at the end:

```

evens = [n for n in range(20) if n % 2 == 0] # only keep n where n % 2 == 0 (no remainder -- means even)

```

And you can transform values conditionally, using the ternary from earlier in this lesson, inside the expression part:

```

labels = ["even" if n % 2 == 0 else "odd" for n in range(5)]
# ["even", "odd", "even", "odd", "even"]

```

Dict comprehensions and set comprehensions work identically, just with `{}` instead of `[]`, and, for dicts, a **key: value** pair instead of a single expression — you'll use these more once you've properly met dicts and sets tomorrow (Day 3), but here's a preview:

```
squares_map = {n: n * n for n in range(5)}    # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

A word of caution, as a beginner: comprehensions are popular in Python because experienced programmers find them fast to read once they're used to the pattern — but as a beginner, if a comprehension is starting to feel like a puzzle you have to squint at, **write it as a normal, multi-line `for` loop instead**. A correct, readable loop beats a clever one-liner you can't quite explain. You'll develop a feel for when a comprehension is genuinely clearer over the next two weeks; there's no rush to force it before then.

`match` — Python's modern multi-way branch (Python 3.10+)

For situations with many possible exact values to check against (similar to a long chain of `elif`, but often clearer to read), Python 3.10 introduced `match`:

```
def describe_day(day_number):
    match day_number:
        case 1:
            return "Monday"
        case 2:
            return "Tuesday"
        case 6 | 7:
            return "Weekend!"
        case _:
            return "Unknown day"
```

the | here means "either of these values"

`\_` is the catch-all, like "else" -- matches anything

`match` can do more than this simple example shows (it can pull apart, or "destructure," more complex data like tuples — you'll see that in the exercises below), but for now, think of it as a cleaner alternative to a long `if/elif/elif/.../else` chain when you're comparing one value against several specific possibilities.

Exercises

Open `exercises.py`, implement each function at the `# TODO` markers, and run it with `python exercises.py` to check your work against the PASS/FAIL output.